TP1: introduction à la programmation avec Java 11

Introduction

Les objectifs de ce TP sont de commencer à se familiariser avec la syntaxe de base du langage Java et de manipuler des fonctions statiques opérant sur des types primitifs. Il s'agit ici de pratiquer un style de programmation *non orientée objet* (i.e., avec des fonctions statiques) de manière à pouvoir la distinguer des pratiques *orientées objet* (OO) qui seront présentées par la suite.

Préparation

Nous vous recommendons de travailler dans un environnement GNU/Linux. Si vous utlilsez Windows, il est possible d'installer WSL (Windows Subsystem for Linux) de manière à avoir un shell Bash nativement intégré: https://docs.microsoft.com/en-us/windows/wsl/install-win10.

Vous aurez besoin d'installer le JDK (inclut les outils nécessaires au développement d'application Java, e.g., JVM, javac etc.): https://adoptopenjdk.net/. Comme dit en cours, il est important de *ne pas utiliser la version fournie par Oracle*, mais bien la **AdoptOpenJDK**.

En résumé, vous pouvez en principe simplement installer JDK via la ligne de commande suivante (sur Ubuntu):

```
sudo apt-get install openjdk-11-jdk
```

Ensuite, pour verifier si tout s'est bien passé, tapez:

```
java --version
```

et vous devriez avoir un output similaire à:

```
openjdk 11.0.10 2021-01-19
OpenJDK Runtime Environment (build 11.0.10+9-Ubuntu-0ubuntu1.20.04)
OpenJDK 64-Bit Server VM (build 11.0.10+9-Ubuntu-0ubuntu1.20.04, mixed mode, sharing)
```

Compilation avec javac et arguments en ligne de commande

Compilation

Durant ce premier TP, nous n'allons pas encore utiliser d'IDE (Integrated Development Environment) tel qu'Eclipse. Nous allons compiler 'à la main' notre code source .java avec la commande `javac`. Cette dernière appelle le compilateur Java que nous venons d'installer. La syntaxe est la suivante:

```
javac <nomDeVotreFichier.java>
```

Ceci va générer un fichier .class dans le même répertoire que votre code source. Pour faire exécuter votre programme par la JVM, la commande est la suivante:

```
java <nomDeVotreFichier>
```

remarquez qu'il ne faut *pas* ajouter l'extension .class.

Exécuter un programme avec des arguments en ligne de commande

Il est possible de passer des arguments via le terminal lorsque vous exécutez votre programme. Ceci peut simplement se faire avec la syntaxe suivante:

```
java <nomDeVotreFichier> arg0 arg1 arg2 ... argn
```

chaque argument doit être séparé du prochain par un espace. Chaque argument est considéré implicitement comme étant un `String`, i.e., une chaîne de caractères, et est stocké dans un tableau de `Strings` comme nous allons le voir par la suite. Si les arguments repérentent sémantiquement des valeurs numériques (à interpréter comme des `float` par exemple), il faudra faire le traitement approprié dans le code, i.e., 'caster' en le type approprié.

Code fourni sur Moodle

Pour ce TP, nous vous fournissons un squelette minimal que vous pouvez télécharger sur Moodle. L'archive contient deux fichier .java:

- TP1Utils.java
- TP1.java

Le premier va contenir des fonctions statiques que nous allons appeler dans TP1. Ce dernière contient la méthode `main` qui est le point d'entrée de notre programme Java. Le code nécessaire à la gestion des arguments en ligne de commande vous ai fourni.

Exercice 1

Dans cet exercice vous allez passer une liste de notes à votre programme Java. Il pourrait s'agir de notes d'étudiants, e.g., une série de valeurs numériques entre 1.0 et 6.0. Il semblerait donc approprié d'utiliser le type `double` ou `float` pour les représenter. Nous vous avons déjà fourni la fonction `str2double(...)` qui va se charger de récupérer les valeurs que vous avez passé en ligne de commande et les convertir en un tableau de `double`.

Soit x un tableau de valeurs numériques (les notes) et n sa taille. Nous vous demandons d'écrire les fonctions suivantes:

1. Une fonction prend x et n en arguments et retourne la **moyenne** des notes: $\overline{x} = \frac{\sum_{i=1}^{n} x_i}{n}$
2. Une fonction qui prend x et n en arguments et retourne **l'écart-type**: $\sigma = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \overline{x})^2}{n}}$
3. Une fonction qui prend x et n en arguments et retourne la **médiane**.
   - Pour rappel, la médiane est la valeur qui sépare une liste triée de valeurs en deux. Il vous faudra donc aussi écrire une fonction qui va *trier* x avant de pouvoir trouver l'élément central. (Vous pouvez utiliser un algorithme de tri de votre choix, le plus simple étant probablement *le tri à bulles*).
   - Si la liste est impaire l'élément central se trouvera à la position $\lceil \frac{n}{2} \rceil$ sinon il faut faire la moyenne de $\frac{n}{2} - 1$ et $\frac{n}{2}$.

Remarque: la fonction ceiling $\lceil . \rceil$ peut être appellée via `Math.ceil(.)`.